

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
.Net Core and Progressive Web Application

Practical – IX

Roll No.: T013	Name: Saqlain Khan
Class: TYIT	Batch: 1
Date of Assignment: 18/08/2026	Date/Time of Submission: 18/08/2026

A. Develop CRUD operations using ASP.NET Core Web API and Entity Framework Core.

ALGORITHM:

Open Visual Studio and create an **ASP.NET Core Web API** project using **.NET 8** with OpenAPI support.

Install the required **Entity Framework Core, SQL Server, Tools, Design and Swagger** packages.

1. Microsoft.EntityFrameworkCore
2. Microsoft.EntityFrameworkCore.SqlServer
3. Microsoft.EntityFrameworkCore.Tools
4. Swashbuckle.AspNetCore
5. Microsoft.EntityFrameworkCore.Design

Create a **Models** folder and add Student.cs with Id, Name, Age and Course properties.

Create a **Data** folder and add ApplicationDbContext.cs with DbSet<Student>.

Add the **LocalDB connection string** for StudentDB in appsettings.json.

Configure ApplicationDbContext, SQL Server and Swagger in Program.cs.

Create StudentsController and implement **GET, GET by ID, POST, PUT and DELETE** operations.

Open **Package Manager Console** and run:

- Add-Migration InitialCreate
- Update-Database

Run the project and open **Swagger UI** to test all CRUD operations.

Open **SQL Server Object Explorer** → **LocalDB** → **StudentDB** → **Tables** → **dbo.Students** and view the inserted student records.

CODE:

(StudentController.cs):

```
using Microsoft.AspNetCore.Mvc;
using Practical_9A.Data;
using Practical_9A.Models;

namespace Practical_9A.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentController : ControllerBase
    {
    }
```

```
private readonly AppDbContext _context;

public StudentController(AppDbContext context)
{
    _context = context;
}

// GET ALL
[HttpGet]
public IActionResult GetResult()
{
    return Ok(_context.Students.ToList());
}

// GET BY ID
[HttpGet("{id}")]
public IActionResult GetStudent(int id)
{
    var student = _context.Students.Find(id);

    if (student == null)
        return NotFound();

    return Ok(student);
}

// INSERT
[HttpPost]
public IActionResult AddStudent(Student student)
{
    _context.Students.Add(student);
    _context.SaveChanges();

    return Ok(student);
}

// UPDATE
[HttpPut("{id}")]
public IActionResult UpdateStudent(int id, Student student)
{
    var data = _context.Students.Find(id);

    if (data == null)
        return NotFound();

    data.Name = student.Name;
    data.Age = student.Age;
    data.Course = student.Course;

    _context.SaveChanges();

    return Ok(data);
}

// DELETE
[HttpDelete("{id}")]
public IActionResult DeleteStudent(int id)
{
    var student = _context.Students.Find(id);

    if (student == null)
        return NotFound();

    _context.Students.Remove(student);
}
```

```

        _context.SaveChanges();
        return Ok();
    }
}

```

(Program.cs):

```

using Microsoft.EntityFrameworkCore;
using Practical_9A.Data;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

(appsettings.json):

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\MSSQLLocalDB;Database=StudentDB;Trusted_Connection=True;"
  }
}

```

(AppDbContext.cs):

```

using Microsoft.EntityFrameworkCore;
using Practical_9A.Models;
namespace Practical_9A.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        public DbSet<Student> Students { get; set; }
    }
}

```

(Student.cs):

```

namespace Practical_9A.Models
{
    public class Student

```

```
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public int Age { get; set; }  
    public string Course { get; set; }  
}
```

OUTPUT:

POST /api/Student

Parameters: No parameters

Request body: application/json

```
{  
  "name": "Himani",  
  "age": 20,  
  "course": "Bsc.IT"  
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \  
  'https://localhost:44376/api/Student' \  
  -H 'accept: */*' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "name": "Himani",  
    "age": 20,  
    "course": "Bsc.IT"  
  }'
```

Request URL: https://localhost:44376/api/Student

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Himani", "age": 20, "course": "Bsc.IT" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 18 Aug 2020 03:25:46 GMT server: Microsoft-IIS/10.0 x-powered-by: ASP.NET</pre>

Responses

Code	Description	Links
200	OK	No links

GET /api/Student/{id}

Parameters

Name	Description
id * required integer(int32) (path)	3

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:44376/api/Student/3' \
  -H 'accept: */*'
```

Request URL

https://localhost:44376/api/Student/3

Server response

Code

Details

200

Response body

```
{
  "id": 3,
  "name": "Himani",
  "age": 20,
  "course": "Bsc.IT"
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 18 Aug 2020 03:26:33 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code

Description

Links

200

OK

No links

PUT /api/Student/{id}

Parameters

Cancel

Reset

Name

Description

id required

integer(\$int32)

(path)

2

Request body

application/json

Request body

application/json

Edit Value

Schema

```
{
  "id": 2,
  "name": "Shubham",
  "age": 22,
  "course": "Btech.EXTC"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  'https://localhost:44376/api/Student/2' \
  -H 'accept: */*' \
  -H 'Content-type: application/json' \
  -d '{
    "id": 2,
    "name": "Shubham",
    "age": 22,
    "course": "Btech.EXTC"
  }'
```

Request URL

https://localhost:44376/api/Student/2

Server response

Code

Details

200

Response body

```
{
  "id": 2,
  "name": "Shubham",
  "age": 22,
  "course": "Btech.EXTC"
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 18 Aug 2020 03:27:24 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code

Description

Links

200

OK

No links

DELETE /api/Student/{id}

Parameters

Cancel

Name

Description

id required

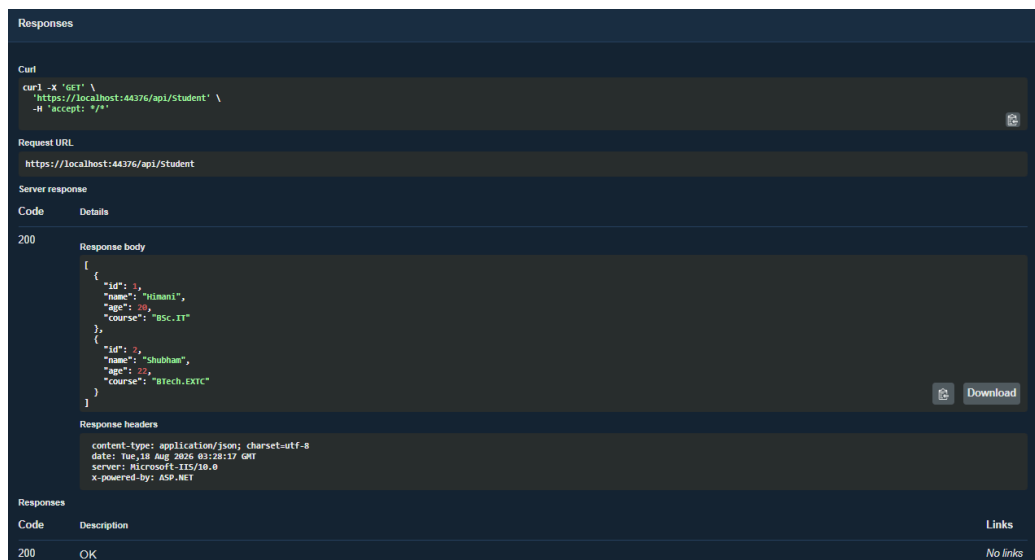
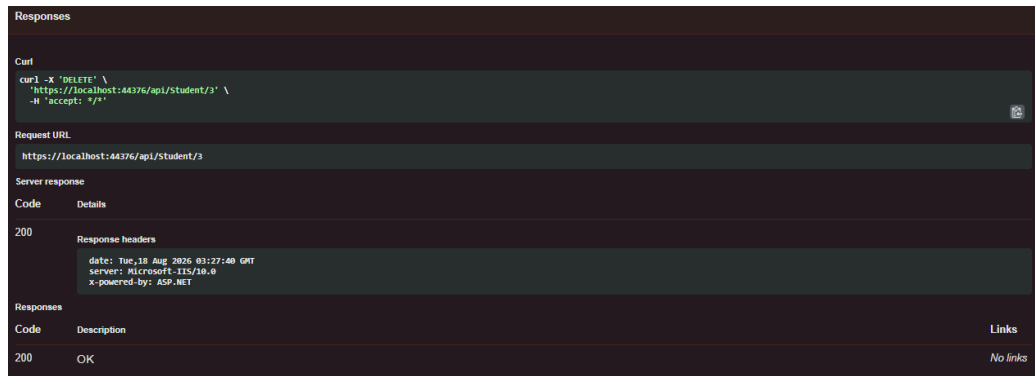
integer(\$int32)

(path)

3

Execute

Clear



View → Sql Server Object Explorer → localdb → Databases → StudentDb → Tables → db0.Student(right click) → View Data

dbo.Students [Data] StudentController.cs Program.cs

Max Rows: 1000

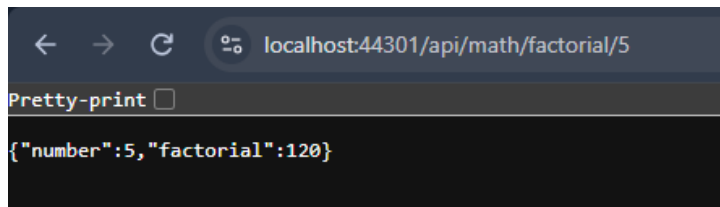
	Id	Name	Age	Course
▶	1	Himani	20	BSc.IT
	2	Shubham	22	BTech.EXTC
	4	Ritesh	21	BCA
⚙	NULL	NULL	NULL	NULL

B. Create an API endpoint to calculate Factorial of a Number and test it using Postman.

CODE:

```
using Microsoft.AspNetCore.Mvc;
namespace Practical_9B.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult GetFactorial(int number)
        {
            if (number < 0)
            {
                return BadRequest("Factorial not possible for -ve nos.");
            }
            long factorial = 1;
            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }
            return Ok(new
            {
                Number = number,
                Factorial = factorial
            });
        }
    }
}
```

OUTPUT:



Verification with Postman:

